Analyzing Loss Value Distribution from Learning Label Noise Data in the Federated Learning Environment

Undergraduate Capstone Project Technical Report

Dohyeok Kwon[†] Seonvin Cho[†] Songnam Hong[*]

Department of Electronic Engineering, Hanyang University

Seoul, Republic of Korea

[†]Equal contribution.

[*]Corresponding author: snhong@hanyang.ac.kr

Abstract

Federated learning can be degraded by mislabeled examples stored on participating clients, while the server cannot inspect the underlying data. This undergraduate capstone project studies whether client-local cross-entropy losses can support noisy-label filtering. Motivated by the observation that clean examples need not form a single low-loss mode, we propose a three-component Gaussian mixture that removes only the component with the largest expected loss. In CIFAR-10 client-local experiments with 25% label corruption, this filter achieved 55.47% test accuracy after four filtering epochs, compared with 28.34% for a two-component Gaussian-mixture baseline. We also formulate a three-component skew-normal mixture to represent asymmetric loss modes. A ten-seed sanity check on the scikit-learn Digits dataset yields $94.71 \pm 1.19\%$ test accuracy and $99.35 \pm 0.29\%$ noise-ranking AUROC. These results support richer client-local loss models, but they do not constitute an end-to-end federated benchmark because the experiments do not evaluate multi-client aggregation and hard selection remains sensitive to thresholding and training stage.

Keywords: federated learning, noisy labels, sample selection, Gaussian mixture, skew-normal mixture

1 Introduction

Federated learning (FL) trains a shared model from decentralized client datasets without directly collecting those datasets at a server. In the standard objective,

$$\min_{w} F(w) = \sum_{k=1}^{K} p_k F_k(w), \qquad p_k = \frac{n_k}{\sum_j n_j}, \tag{1}$$

client k performs local updates from a broadcast model and the server aggregates the returned parameters using data-dependent weights [1]. Label noise is especially difficult in this setting because mislabeled samples remain private to each client and local data distributions can be heterogeneous.

A common defense uses the *small-loss effect*: during early training, deep networks tend to fit cleanly labeled examples before memorizing corrupted labels [2, 3, 6]. Co-teaching exchanges small-loss examples between two networks [3], while DivideMix models normalized per-example losses with a two-component Gaussian mixture [4]. FedRN adapts loss-based selection to FL by combining the target client's model with reliable neighbor models and fitting separate two-component mixtures [5]. Subsequent FL work has introduced multi-stage label correction, standardized noisy-label benchmarks, and collaborative global noise filters [7, 8, 9].

This project asks a narrower question: can a richer loss mixture avoid discarding clean examples that have not yet reached the smallest-loss mode? We propose a three-component local filter, TRIMIX, evaluate it under FL-motivated label-corruption scenarios, and extend it with a skew-normal mixture, TRIMIX-SN, for asymmetric loss distributions.

Scope. The project targets a client-side module that can be inserted before local SGD in an FL round. The experiments train and evaluate a single local model; they do not specify multiple clients, non-IID partitioning, client sampling, or server aggregation. The results therefore provide *client-local evidence for an FL component*, not an end-to-end FL evaluation.

2 Pilot Study

The pilot used CIFAR-10, containing 50,000 training and 10,000 test images from ten classes [10]. The CNN contained two convolutional layers, two pooling layers, and two fully connected layers. Two corruption mechanisms were considered:

1. **Deterministic class mapping:** every training example from true classes 0–4 was relabeled as class $y+5$. Because CIFAR-10 is class-balanced, this corrupts 25,000 labels, or 50% of the training set.

2. **Random wrong label:** every example from true classes 0–4 was assigned a randomly selected incorrect label.

This design compares structured and random corruption, but it also confounds label status with true class: clean examples come from classes 5–9 and corrupted examples from classes 0–4.

For each mechanism, we compared a randomly initialized model with a model pretrained for three clean epochs. We then collected one cross-entropy loss per training example while learning from the corrupted set. Table 1 reports the test accuracies.

Table 1: CIFAR-10 pilot accuracy (%).

	Random init.		3-epoch pretrain	
Corruption	Before	After	Before	After
Mapping	10.27	35.77	64.24	43.48
Random	10.34	35.85	64.73	48.06

The pretrained models lose 20.76 and 16.67 percentage points, respectively, after exposure to corrupted data. The loss histograms in Fig. 1 also show a near-zero mass together with a broader clean-data mode after pretraining. This observation motivated a third mixture component. The histograms alone, however, do not invalidate Gaussian mixtures: that claim would require goodness-of-fit or sample-selection metrics.

3 Three-Component Loss Filtering

At filtering step t, the current client model produces losses

$$\ell_i^{(t)} = -\log p_{w_t}(\widetilde{y}_i \mid x_i) \tag{2}$$

for local examples $(x_i, \widetilde{y}_i)$. TRIMIX fits a three-component univariate Gaussian mixture,

$$q(\ell) = \sum_{j=1}^{3} \pi_j \mathcal{N}(\ell; \mu_j, \sigma_j^2), \qquad \sum_j \pi_j = 1, \tag{3}$$

using expectation-maximization (EM). Let r_{ij} be the posterior responsibility of component j, and let $j^* = \arg\max_j \mu_j$. The retained subset is

$$\mathcal{S}_t = \left\{ i : \arg\max_j r_{ij} \neq j^* \right\}. \tag{4}$$

The client updates its model only on $\mathcal{S}_t$. The two lower-mean components are intended to capture easy clean examples and harder or partially learned clean examples; the largest-mean component is treated as noisy.

The two-component comparator follows FedRN's loss-mixture filtering idea. Full FedRN also retrieves reliable neighbor models, weights their mixture estimates, and fine-tunes them locally [5]; because those operations are not included here, we use the precise label **2-GMM (FedRN-style local filter)**.

4 Experimental Results

The experiments use 25% corrupted labels under two client-local scenarios. The exact corruption kernel, optimizer settings, and random seeds were not documented, so the values below should be interpreted as a project-scale comparison rather than a reproducible benchmark.

Scenario A: corruption after clean pretraining. The model was first trained on 30,000 clean examples and then exposed online to 25% corrupted labels in previously unseen data. The initial test accuracy was 61.26%. After the reported sequence of 5,000-example updates, 2-GMM ended at 36.84%, while TRIMIX ended at 43.65%.

Table 2: Results with 25% label noise (%). "Detect" denotes clean/noisy classification accuracy.

Epoch	Method	Detect	Test acc.
1	2-GMM	39.27	20.24
	TRIMIX	**57.13**	**34.84**
2	2-GMM	42.99	22.77
	TRIMIX	**61.39**	**42.89**
3	2-GMM	43.52	26.15
	TRIMIX	**75.68**	**49.93**
4	2-GMM	48.04	28.34
	TRIMIX	**76.50**	**55.47**

Scenario B: corruption from the start. Table 2 reports the results across four filtering epochs. TRIMIX improves the final test accuracy by 27.13 percentage points over 2-GMM. The clean/noisy classification accuracy reaches 76.50%, but raw accuracy is a weak metric here: with 25% corruption, an all-clean prediction already obtains 75%. Balanced accuracy, precision–recall, and AUROC would be more informative.

5 Skew-Normal Mixture

Cross-entropy losses are nonnegative and often right-skewed, so a symmetric Gaussian component can fit a mode poorly. The skew-normal variant, TRIMIX-SN, replaces each Gaussian in Eq. (3) with a skew-normal density [11]:

$$f_{\mathrm{SN}}(\ell; \xi, \omega, \alpha) = \frac{2}{\omega}\, \phi(z)\Phi(\alpha z), \tag{5}$$

$$z = \frac{\ell - \xi}{\omega}, \qquad \omega > 0, \tag{6}$$

$$q_{\mathrm{SN}}(\ell) = \sum_{j=1}^{3} \pi_j f_{\mathrm{SN}}(\ell; \xi_j, \omega_j, \alpha_j). \tag{7}$$

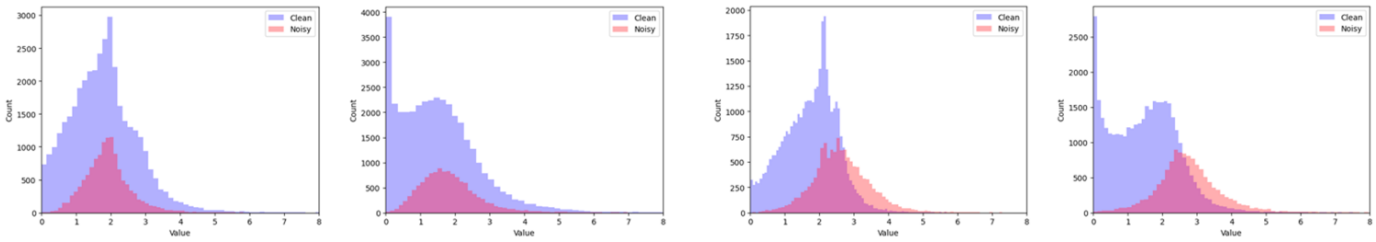
Here ϕ and Φ are the standard-normal density and CDF. The E-step computes the usual mixture responsibilities. In a generalized M-step, π_j has a closed-form update while $(\xi_j, \log \omega_j, \alpha_j)$ maximize the responsibility-weighted log-likelihood numerically. Components are ordered by expected loss

$$m_j = \xi_j + \omega_j \delta_j \sqrt{\frac{2}{\pi}}, \qquad \delta_j = \frac{\alpha_j}{\sqrt{1+\alpha_j^2}}, \tag{8}$$

and samples assigned to $\arg\max_j m_j$ are removed.

5.1 Reproducible Sanity Check

The per-example CIFAR-10 losses were unavailable, so TRIMIX-SN was evaluated separately on the built-in scikit-learn Digits dataset [12]. We used a stratified 75/25 train/test split, exact-rate 25% symmetric label corruption on the training set, a one-hidden-layer MLP with 64 units, two unfiltered warm-up epochs, and five filtering epochs. Results in Table 3 are means and sample standard deviations over ten seeds. The accompanying implementation records every seed-wise metric in a machine-readable result file.



(a) Deterministic class mapping (b) Random wrong labels

Figure 1: Per-example loss histograms. In each panel, the left plot is obtained from a randomly initialized model and the right plot after three clean pretraining epochs. Blue and red denote clean and corrupted examples. Counts, rather than normalized densities, are shown.

Table 3: Ten-seed Digits results with 25% symmetric label noise (mean ± standard deviation, %). Selection metrics are measured after the final filtering epoch.

Metric	None	2-GMM	TRIMIX	TRIMIX-SN
Test acc.	91.02 ± 1.75	91.09 ± 4.87	94.36 ± 1.53	**94.71 ± 1.19**
Selection acc.	–	90.02 ± 3.43	95.03 ± 4.51	**95.18 ± 3.18**
Balanced acc.	–	**92.93 ± 2.48**	92.40 ± 9.92	91.11 ± 6.91
Noise AUROC	–	94.94 ± 2.52	99.04 ± 0.45	**99.35 ± 0.29**

TRIMIX-SN slightly improves mean test accuracy and noise-ranking AUROC over 3-GMM. It does not improve hard-selection balanced accuracy, indicating that better density ranking does not automatically yield a better decision threshold. The result supports skew-aware loss modeling while identifying posterior calibration or adaptive thresholds as the next technical step.

6 Discussion and Limitations

The project contains a useful empirical idea: a clean set can occupy more than one loss regime, so discarding every sample outside a single low-loss component may be unnecessarily aggressive. The three-component rule operationalizes this observation with little added complexity and preserves more intermediate-loss examples.

The evidence remains limited in four ways. First, the study does not run a complete FL loop, so it cannot establish robustness under client heterogeneity or aggregation. Second, the CIFAR-10 accuracy values lack seed-wise uncertainty and complete hyperparameters. Third, high loss is not equivalent to label corruption: difficult but correctly labeled examples can be removed, while memorized noisy examples can migrate to low loss. Fourth, mixture structure is training-stage dependent; the loss histograms show weak clean/noisy separation in the first epoch. Future work should evaluate calibrated noise scores across rounds and report AUROC, area under the precision–recall curve, retained-clean recall, and global-model accuracy under standardized IID and non-IID client partitions [8, 9].

7 Conclusion

This project formulates a client-local loss filtering method for FL. The CIFAR-10 experiments show that retaining two lower-loss components can outperform a two-component local filter under 25% label corruption. The skew-normal mixture provides a principled way to model asymmetric loss modes and shows a small improvement in noise ranking and test accuracy in a separate ten-seed sanity check. The result is best viewed as an undergraduate proof of concept and a reproducible starting point for end-to-end federated evaluation.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.

[2] D. Arpit et al., "A closer look at memorization in deep networks," in *Proc. ICML*, 2017, pp. 233–242.

[3] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. Tsang, and M. Sugiyama, "Co-teaching: Robust training of deep neural networks with extremely noisy labels," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[4] J. Li, R. Socher, and S. C. H. Hoi, "DivideMix: Learning with noisy labels as semi-supervised learning," in *Proc. ICLR*, 2020.

[5] S. Kim, W. Shin, S. Jang, H. Song, and S. Yun, "FedRN: Exploiting *k*-reliable neighbors towards robust federated learning," in *Proc. CIKM*, 2022, pp. 972–981.

[6] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, "Learning from noisy labels with deep neural networks: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 8135–8153, 2023.

[7] J. Xu, Z. Chen, T. Q. S. Quek, and K. F. E. Chong, "FedCorr: Multi-stage federated learning for label noise correction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10174–10183.

[8] S. Liang, J. Huang, J. Hong, D. Zeng, J. Zhou, and Z. Xu, "FedNoisy: Federated noisy label learning benchmark," arXiv:2306.11650, 2023.

[9] J. Li, G. Li, H. Cheng, Z. Liao, and Y. Yu, "FedDiv: Collaborative noise filtering for federated learning with noisy labels," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 4, 2024, pp. 3118–3126.

[10] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Technical Report, 2009.

[11] A. Azzalini, "A class of distributions which includes the normal ones," *Scandinavian Journal of Statistics*, vol. 12, no. 2, pp. 171–178, 1985.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.